

ADVANCED SOFTWARE ENGINEERING

Abdulfattah Alfarra

استراتيجية التنمية

OUTLINE

□ التعريف

□ البرمجة الزوجية □ الملكية

□ الجماعة □ معيار الكود

40 □ ساعة في الأسبوع

□ إعادة الهيكلة

Definition

ربت عي الكود هو جوهر أي برنامج.
كلما كان الكود أفضل، كلما كان رد الفعل على المدخلات
أفضل.

ترشح البرمجة المتطرفة الترميز باعتباره **النشاط الرئيسي**.

جم ر ب م ل هو قلب XP.

في نظام التشغيل ، XP تقوم بالبرمجة لأنه إذا لم تقم بالبرمجة، فلن تكون
قد فعلت أي شيء في نهاية اليوم.

Definition

الروتين اليومي لمطور XP

إقران



حدد المهمة



تصميم سريع



كتابة الاختبار



شفرة



إعادة الهيكلة



دمج



العودة إلى المنزل

Definition

يعمل **المطورون** في أزواج، ويتشاركون كل محطة عمل.

قد يكون زوجك محددًا بالفعل إذا كنت لا تزال تعمل على مهمة ما، وإلا فسوف تحتاج إلى العثور على مطور آخر من الفريق للتعاون معه.

تحديد المهمة على افتراض أنك لم تعد تعمل على مهمة ما، فسوف يحتاج زوجك إلى تحديد مهمة الهندسة التالية للعمل عليها.

Definition

التصميم السريع يعني التصميم في هذا المستوى "كيف سنحل هذه المهمة؟"

ابدأ **المهمة** بكتابة حالة اختبار، وتأكد من فشلها قبل الشروع في البرمجة. وسّع نطاق اختبارك ليشمل كل ما قد يفشل. اطلب توضيحات من فريق خدمة العملاء عند الحاجة.

Definition

الكود: اكتب الكود لإكمال المهمة. الهدف هو كتابة ما يكفي من الكود لضمان اجتياز المكون لحالة الاختبار. **أعد** صياغة الكود. راجع الكود، وابحث عن جوانب التبسيط والتحسين. توقف عن إعادة الصياغة عند تحقيق البساطة.

Definition

دمج: أضف المكون كاملاً إلى جهاز التكامل أو البناء. تأكد من اجتياز الفئة المدمجة للاختبارات. إذا استمرت مشاكل التكامل، فتخلص من الكود وابدأ من جديد غداً.

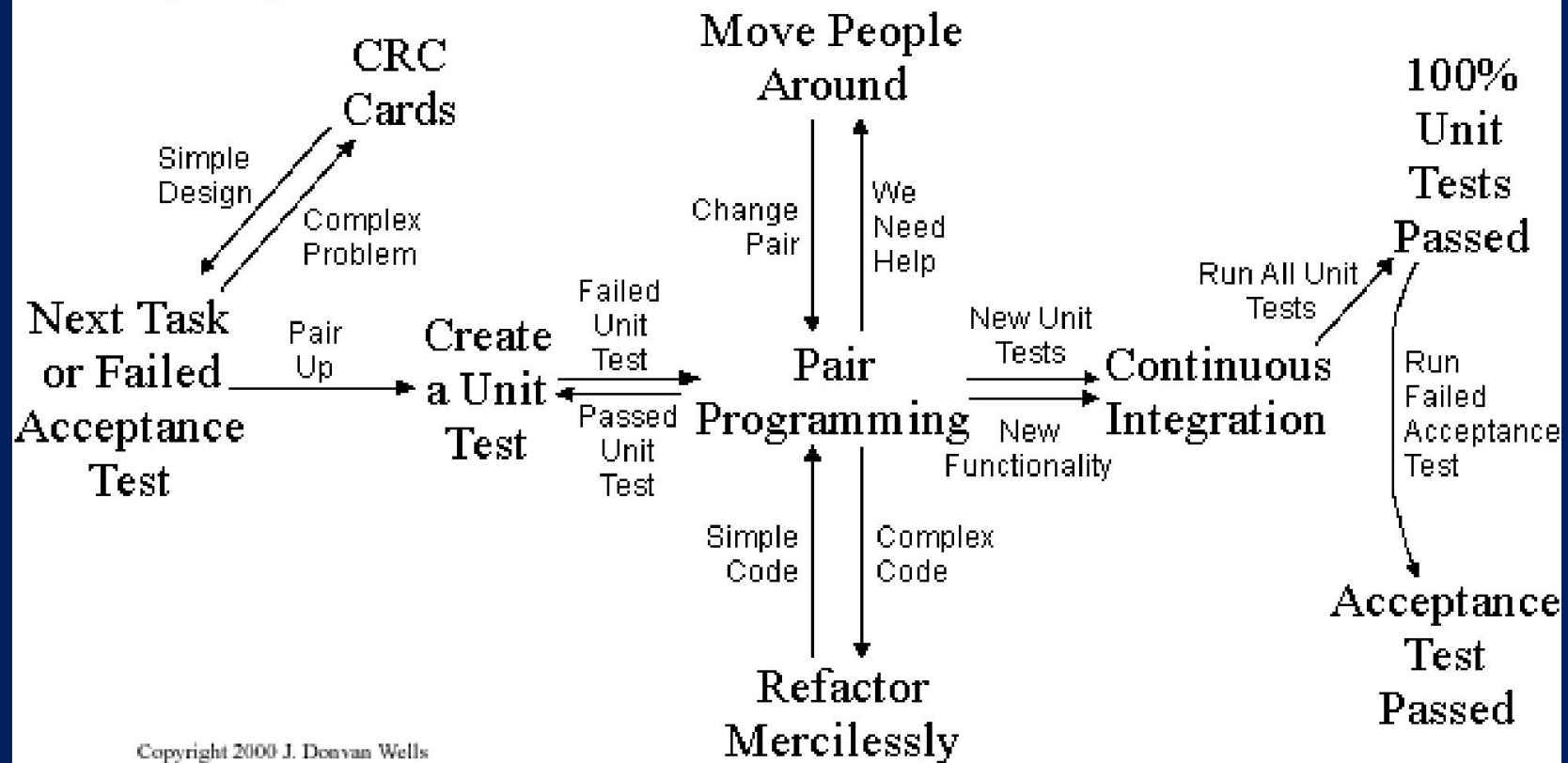
العودة إلى المنزل تمشياً مع ممارسة XP المتمثلة في العمل لمدة 40 ساعة في الأسبوع، العودة إلى المنزل في الساعة 5 مساءً

XP Project Coding



Collective Code Ownership

Zoom Out



Copyright 2000 J. Donovan Wells

Pair Programming

□ يتعاون اثنان من المبرمجين على نفس التصميم أو الخوارزمية أو الكود
أو حالة الاختبار □ الاقتران ديناميكي □ يشجع التواصل والإنتاجية ويعزز
جودة الكود

□ يعتبر الاقتران مفيدًا للتدريب المتبادل للموظفين الحاليين وللتدريب
للموظفين الجدد



Pair Programming

قواعد:

□ تتم جميع جلسات البرمجة "جنبًا إلى جنب": شخصين على شاشة واحدة □ يجب أن يفهم كلاهما كل سطر من التعليمات البرمجية ويتفقان عليها .

□ قم بتبديل السائقين من وقت لآخر.

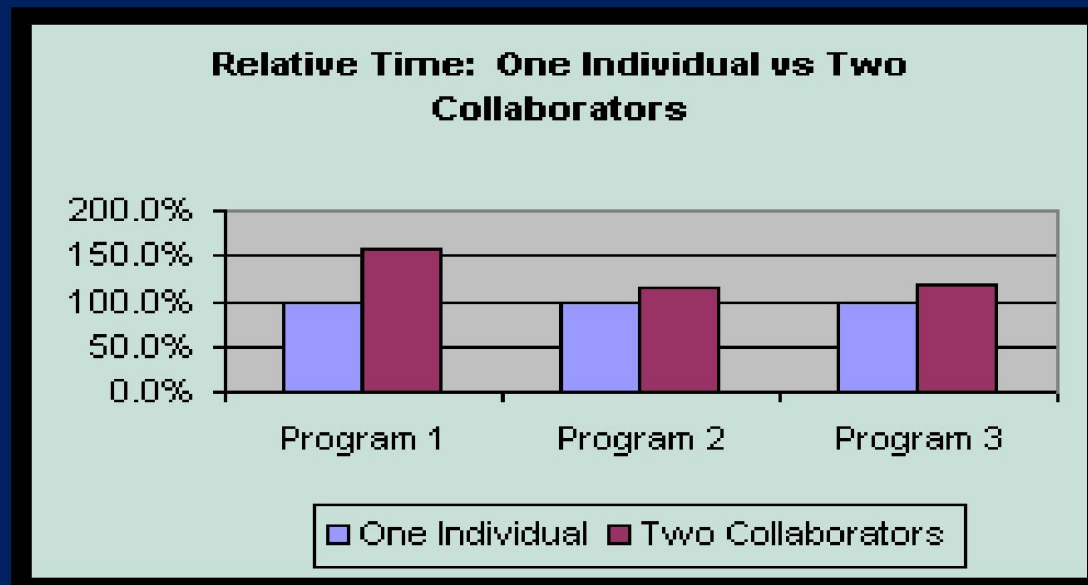
Pair Programming

يكشف بحث البرمجة الزوجية أن: □ الأزواج لا يستخدمون ساعات عمل أكثر من الأفراد
□ الأزواج يخلقون عيوبًا أقل

□ الأزواج تنشئ عددًا أقل من أسطر التعليمات البرمجية
□ يستمتع الأزواج بعملهم أكثر

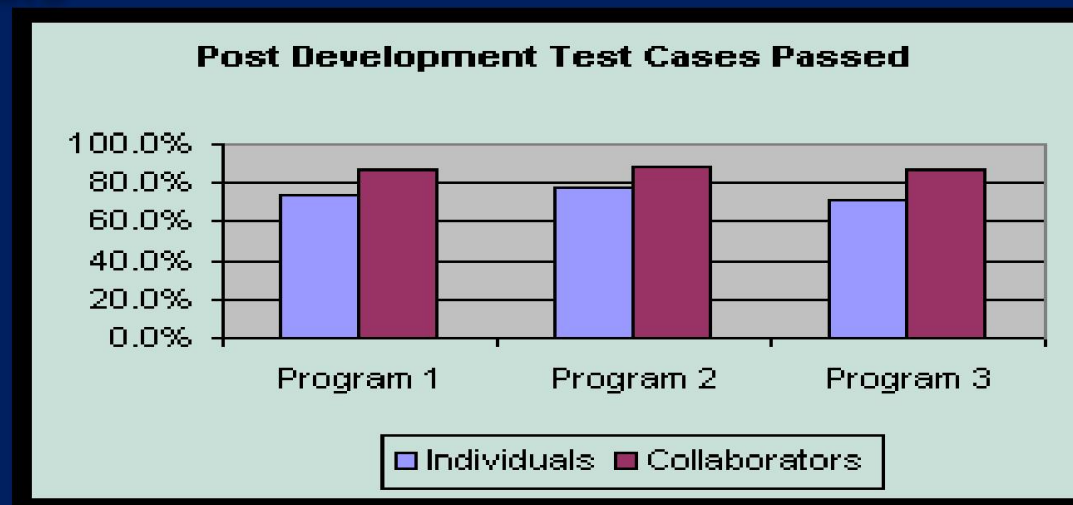
برمجة الزوج

University of Utah Experiment: Pairs spent 15% more time on the program than individuals



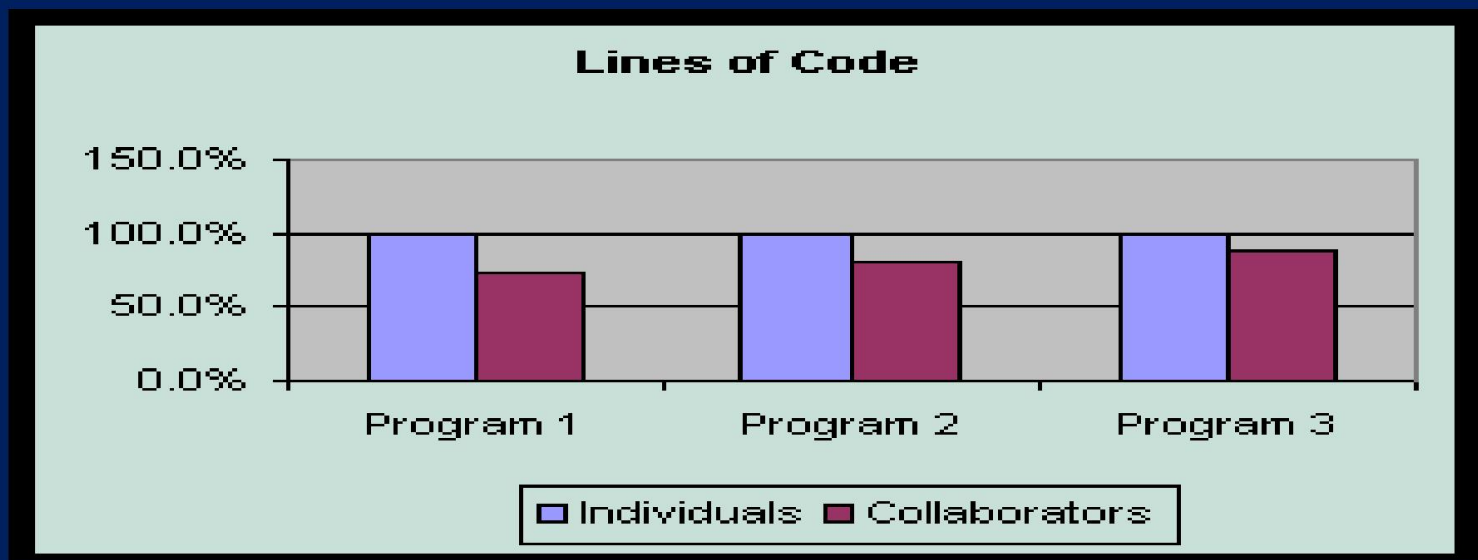
برمجة الزوج

University of Utah Experiment: Code written by pairs passed more test cases than code written by individuals



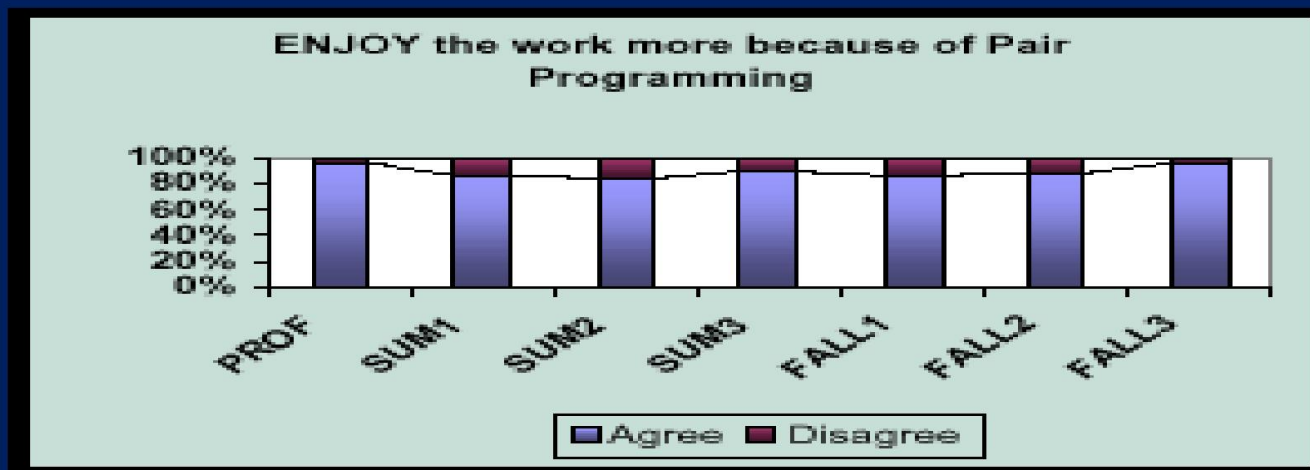
برمجة الزوج

University of Utah Experiment: Pairs consistently implemented the same functionality produced by individuals in fewer lines of code



برمجة الزوج

University of Utah Experiment: Learning how to program in an environment where there are rapidly tangible results is fun and allows one to learn faster



برمجة الزوج

تجربة جامعة يوتا: الاستنتاجات

الفوائد المهمة للبرمجة الزوجية هي:

□ يتم اكتشاف العديد من الأخطاء أثناء كتابتها وليس أثناء الاختبار.

□ محتوى العيب النهائي أقل إحصائيًا □ التصميمات أفضل □ طول الكود أقصر □

يحل الفريق المشكلات بشكل أسرع

Pair Programming

عيوب البرمجة الزوجية : اختلاف المهارات: هذه هي المشكلة المحتملة الأولى. إذا كان مستوى مهارات الشريكين مختلفًا تمامًا، فقد يتولى أحد المبرمجين العمل بأكمله أو يُدرَّب الآخر باستمرار. لا بأس بذلك إذا كنت ترغب في بناء علاقة بين المعلم والطالب أو كنت تُعرِّف مبرمجًا جديدًا على نظامك ، ولكن إن لم يكن كذلك، فقد يُفسد هذا الغرض من البرمجة الزوجية.

Pair Programming

عدم إنجاز العمل فعليًا: بالنسبة للبعض ، قد تُنتج جلسات البرمجة الثنائية بسهولة جلسات تواصل اجتماعي. هناك من لا يعمل عندما يكون هناك من يراجع عمله بجانبه، ولن يستفيدوا كثيرًا أيضًا.



Pair Programming

غرور المطورين : من غير المرجح أن يحدث هذا في بيئة الفصل الدراسي، ولكن في الفرق الأكثر خبرة، قد يحاول كل مبرمج فرض أفكاره الخاصة حول كيفية إنجاز الأمور (وكلاهما قد يكون صحيحًا تمامًا). قد يكون هذا النوع من الصراعات كارثيًا.

Pair Programming

□ وقت بدء التشغيل: هناك حاجة إلى بعض وقت بدء التشغيل للتكيف مع البيئة المقترنة.

Pair Programming

تقييم **المبرمجين**: يُصبح تقييم الأفراد ودفع أجورهم بناءً على مساهماتهم أمرًا بالغ الصعوبة. في سياق الدورات الدراسية، يُمكن التغلب على هذه المشكلة بطرق عديدة. أما في العالم الصناعي، فتُمثل هذه المشكلة مشكلة حقيقية دون حل واضح.

Pair Programming

لقد تفاجأ الكثيرون بالبرمجة الزوجية. جرّبوها، متوقعين أن يكرهوها، لكنهم وجدوا أنها أعجبتهم حقًا.

Collective Ownership

ماذا يحدث عندما يذهب شخصٌ ناقدٌ في إجازة، أو يبقى في المنزل مع طفلٍ مريض، أو يحصل على وظيفةٍ جديدة، أو يتقاعد فجأةً؟ كم من الوقت ستقضيه في تدريب بديل؟

Collective Ownership

الملكية الجماعية للشفرة البرمجية تُوزَّع مسؤولية صيانة الشفرة على جميع المبرمجين. الملكية الجماعية للشفرة البرمجية هي تمامًا كما يوحى اسمها: يتشارك الجميع مسؤولية جودة الشفرة. لا يدّعي أي شخص ملكية أي جزء من النظام، ويمكن لأي شخص إجراء أي تغييرات ضرورية في أي مكان.

Collective Ownership

□ يمكن لأي عضو في الفريق الإضافة إلى الكود في أي وقت
وقت

□ يتحمل الجميع المسؤولية الكاملة
النظام □ يشجع البساطة: □ يمنع دخول الكود المعقد إلى النظام □ يزيد من المسؤولية
الفردية والشخصية

الطاقة □ تقلل من مخاطر المشروع:

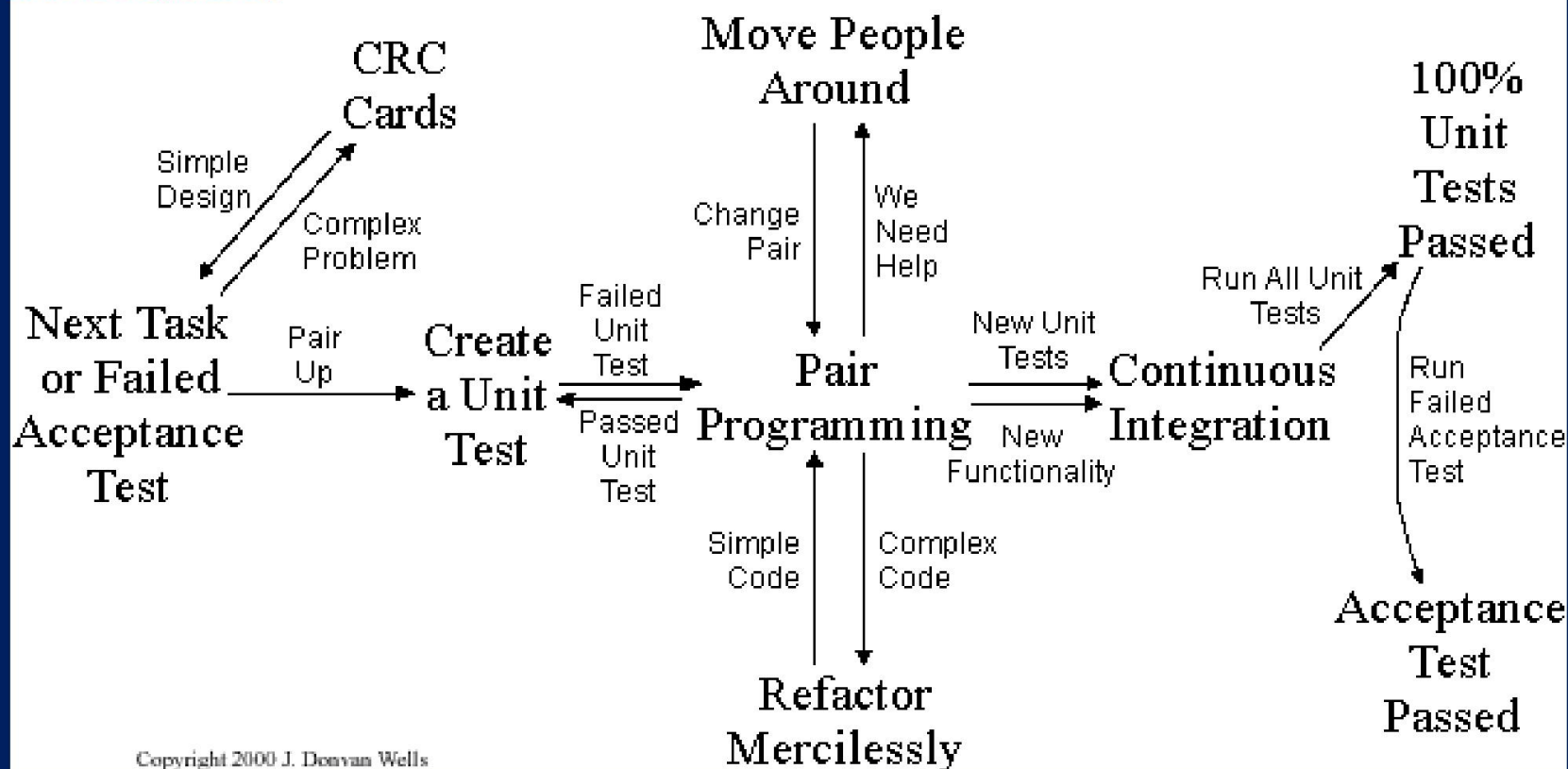
□ ينشر المعرفة بالنظام بين أعضاء الفريق

XP Project Coding



Collective Code Ownership

Zoom Out



Copyright 2000 J. Donovan Wells

Collective Ownership

جعل الملكية الجماعية تعمل تتطلب التخلي عن القليل من الأنا: بدلاً من أن
تفخر بالكود الخاص بك، افتخر بكود فريقك.

بدلاً من الشكوى عندما يقوم شخص ما بتحرير الكود الخاص بك، استمتع
بكيفية تحسين الكود عندما لا تعمل عليه.

Collective Ownership

بدلاً من دفع رؤيتك التصميمية الشخصية، ناقش إمكانيات التصميم مع المبرمجين الآخرين
واتفق على حل مشترك.

Collective Ownership

الفوائد □ فرق تطوير البرمجيات غير مستقرة إلى حد كبير: بالنسبة لنظام التشغيل XP، يبدو أن فرق البرمجيات تتغير باستمرار.

انضمام أشخاص باستمرار، ومغادرة الفريق، والمرض، أو التعرض للدهس من قبل شاحنة. ببساطة، لا يُمكن الاعتماد على شخص واحد للبقاء لفترة أطول لتكليفه بمسؤولية أي جزء من المشروع. □ **المعرفة المشتركة بالشفرة البرمجية:** تُتيح للمبرمجين التعرّف على المزيد من الشيفرة البرمجية والاستفادة من تجارب الآخرين.

Collective Ownership

□ **كود أبسط:** يؤدي إلى العثور على الكود المعقد وإعادة صياغته بسرعة أكبر حيث يقرأ العديد من أزواج العيون نفس الكود.

□ **إنجاز الأمور بسرعة:** إزالة العقبات حتى يتمكن أولئك الذين يحتاجون إليها من إجراء التغييرات عندما يحتاجون إليها.

Collective Ownership

تحسين جودة الكود. تتيح الملكية الجماعية للجميع إصلاح المشاكل التي يجدونها. إذا واجهت تكرارًا، أو أسماء غير واضحة، أو حتى كودًا سيئ التصميم، فلا يهم من كتبه. إنه كودك . أصلحه!

Collective Ownership

عيوب الملكية الجماعية : هناك خطر أن يصبح شخص واحد في المكتب مسؤولاً عن إصلاح الأخطاء. لا يستمتع الكثير من المطورين بإصلاح الأخطاء طوال اليوم، لذا من المهم، برأيي، التأكد من تقاسم مسؤولية إصلاح الأخطاء بالتساوي بين فريق التطوير.

Collective Ownership

إذا قام المزيد من الأشخاص بتغيير أي منطقة من قاعدة التعليمات البرمجية، فمن المرجح أن تحدث تعارضات مباشرة في التعليمات البرمجية (مجموعات التغييرات المتضاربة). غالبًا ما يكون الأشخاص المسؤولون عن الإنشاء الأولي لوحدة معينة غير راغبين في تركها وتعديلها بواسطة مطورين آخرين.

Coding Standards

تُوضّح وثيقة معايير البرمجة للمطورين كيفية كتابة أكوادهم. فبدلاً من أن يكتب كل مطور أكواده بأسلوبه المفضل، سيكتبون جميع أكوادهم وفقاً للمعايير الموضحة في الوثيقة.

يضمن هذا تناسق أسلوب برمجة أي مشروع كبير ، فلا يكتب مبرمجون مختلفون أجزاءً مختلفة. لا يُسهّل هذا الحل فهم الكود فحسب، بل يضمن أيضاً أن أي مطور يقرأ الكود سيعرف ما يتوقعه خلال التطبيق بأكمله.

Coding Standards

إذا كنت تستخدم نظام ، XP فستُطوّر برامجك في أزواج، كلٌّ يتناوب على لوحة المفاتيح. قبل بدء البرمجة، ستحتاج إلى مجموعة قواعد مُتفق عليها حول كيفية إدارة التطوير. هذا يُجنّبك الجدل التافه حول التنسيق والتسمية وما إلى ذلك. بدون معايير البرمجة ، سيصبح تبادل الأزواج صعبًا لأنك ستُجبر على تعلم كل أسلوب ونهج مُطوّر آخر. لذا ، فإن معايير البرمجة ليست مجرد أفضل الممارسات، بل هي تُسهّل عمل فريق التطوير وتضمن فوز الجميع.

Coding Standards

يجب أن يتطلب المعيار أقل قدر ممكن من العمل، بما يتوافق مع قاعدة "مرة واحدة فقط" (لا تكرار للكود). يجب أن يُركز المعيار على التواصل.

وأخيرا، يجب أن يتم اعتماد المعيار طوعية من قبل الفريق بأكمله.

Coding Standards

□ معايير الترميز هي إرشادات لأسلوب الترميز والتوثيق.

□ الحلم هو أن أي مطور على دراية بالمبادئ التوجيهية يمكنه العمل على أي كود يتبعها.

□ تتراوح المعايير من سلسلة بسيطة من البيانات إلى المستندات المعنية.

□ ينبغي أن يتم الترميز بهدف الصيانة.

Coding Standards

□ حدد المعايير التي تطبقها المعيار الكامل

technetwork/java/codeconvtoc-136057.html

<http://www.oracle.com/>

40-hour week

في بيئة XP، يتميز التطوير بسرعة فائقة وتواصل عالي. يعمل المطورون في بيئة يسودها **ضغط** التغيير. العمل على مشاريع XP يعني الارتقاء المستمر بالجودة والأداء طوال عمر المشروع.

40-hour week

كيف تحافظ على الجودة مع فرق العمل التي تعمل لساعات إضافية كثيرة؟ **الإجابة هي لا يمكنك ذلك.** مع الأخذ في الاعتبار أن مبدأ الخبرة الأساسي هو أن جودة العمل لا يمكن المساس بها، سيتعين علينا التحكم في عدد ساعات عمل المطورين مع الحفاظ على معايير عالية. إذا أمكن توجيه الطاقة حصرًا نحو المشروع لمدة ثماني ساعات يوميًا، مع تجاهل أي شيء خارجه، فقد يكون **العمل لمدة 40 ساعة** أسبوعيًا ناجحًا.

40-hour week

فكرة أسبوع العمل المكون من 40 ساعة هي أن يعمل أعضاء الفريق الساعات التي تسمح لهم بالحفاظ على الجودة. يمكن أن يكون عبء العمل المستدام 40 ساعة أو نحو ذلك.

40-hour week

□ إن إرساء سياسة ساعات العمل المستدامة يؤدي أيضًا إلى إزالة "السوق السوداء" في وقت المشروع.

يشير هذا إلى الممارسة المشبوهة المتمثلة في تثبيت أسعار العقود (مما يُلزم الفريق فعليًا بأجر لا يتجاوز ثماني ساعات يوميًا)، مع السماح للإدارة في الوقت نفسه بالضغط على الفريق للعمل لساعات أطول دون أجر. أما الموظفون الذين يتقاضون رواتبهم، فهم يضيعون وقتًا شخصيًا دون أي مكافأة.

40-hour week

قاعدة الخبرة بسيطة: لا يمكنك العمل الإضافي لأسبوعٍ ثانٍ . لمدة أسبوع واحد، حسنًا، حسن أدائك وزد ساعات عملك. إذا أتيت يوم الاثنين وقلت: "لنحقق أهدافنا، علينا العمل لساعات متأخرة مجددًا"، فأنت تواجه مشكلة لا يمكن حلها بزيادة ساعات العمل.

Refactoring

ما هو إعادة الهيكلة؟

إعادة الهيكلة هي تقنية تحسين الكود دون تغيير الوظيفة. إعادة الهيكلة هي عملية تبسيط مستمرة تُطبَّق على الكود والتصميم والاختبار وتجربة المستخدم نفسها.

Refactoring

إليك إعادة هيكلة بسيطة. لنفترض أن لديك الكود التالي:

الفئة العامة Person { سلسلة خاصة

firstName;

public void setFirst(String n) {

هذا الاسم الأول = n;

}

Refactoring

يمكن تحسين هذا الكود عن طريق اختيار اسم وسيطة أفضل، وتغيير n إلى : firstName

الفئة العامة { Person

{ this.firstName = firstName; } سلسلة خاصة

public void setFirst(String firstName)
firstName;

}

Refactoring

يمكن تحسين الكود بشكل أكبر عن طريق إعادة تسمية الطريقة
إلى : `setFirstName()`

```
الفئة العامة Person { سلسلة خاصة  
public void  
= firstName! } emaNtsriFtesfirstName!  
firstName) { this.firstName  
  
}
```


Refactoring

لماذا يجب أن يتم إعادة الهيكلة؟

(إعادة الهيكلة تُحسّن تصميم البرمجيات. فبدون إعادة الهيكلة، سيتدهور تصميم البرنامج. فعندما يُجري الناس تغييرات على الكود -سواءً لتحقيق أهداف قصيرة المدى أو دون فهم كامل لتصميم الكود -يفقد الكود بنيته.

Refactoring

(إعادة الهيكلة تجعل البرنامج أسهل للفهم. - هناك مستخدمون للكود الخاص بك.

الكمبيوتر والكاتب والمحدث.
الأهم هو المحدث. لا يهم إن استغرق المجمع بضع دورات إضافية لتجميع الكود. إذا استغرق تحديث الكود ثلاثة أسابيع، فهذه مشكلة.

Refactoring

(3) تساعدك إعادة الهيكلة على العثور على الأخطاء. - جزء من إعادة هيكلة الكود هو فهم الكود ووضع هذا الفهم مرة أخرى في الكود.

في هذه العملية، يتم توضيح الأخطاء.

Refactoring

(إعادة الهيكلة تساعدك على البرمجة بشكل أسرع: التصميم الجيد ضروري للحفاظ على سرعة تطوير البرمجيات. إعادة الهيكلة تساعدك على تطوير البرمجيات بشكل أسرع، لأنها تمنع تدهور تصميم النظام.

ويمكنه أيضًا تحسين التصميم.

Refactoring

خصائص إعادة الهيكلة → التحويل من مصدر إلى مصدر

→ البقاء داخل نفس اللغة، على سبيل المثال، Java إلى جافا

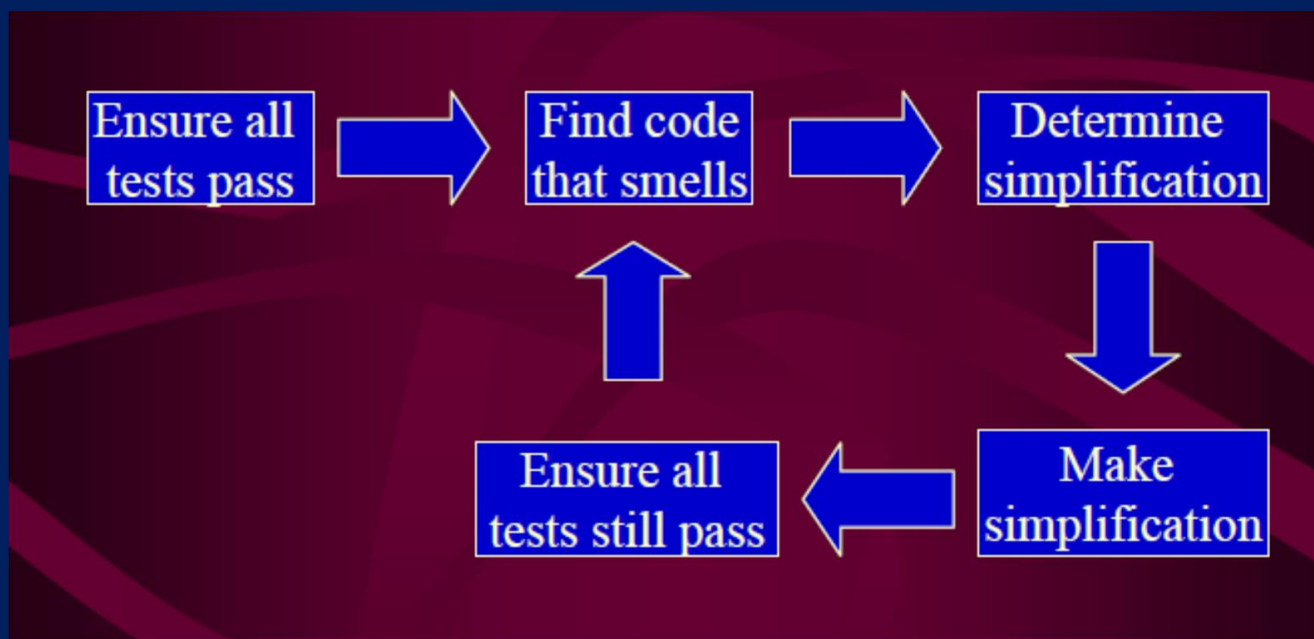
→ لا يغير سلوك البرامج → تم تصميمه في الأصل للغات الموجهة للكائنات، ولكن يمكن تطبيقه أيضًا

على ميزات اللغة غير الموجهة للكائنات، أي الوظائف

Refactoring

"رائحة الكود" تعني أن هناك مؤشرات تشير إلى كود سيئ .

تتضمن عملية إعادة الهيكلة الخطوات البسيطة التالية:



Refactoring

رائحة الكود

□ مؤشرات تشير إلى أن هناك خطأ ما في
شفرة

□ يمكن أن يحدث في كل من كود الإنتاج والاختبار
شفرة

Code Smells

-الكود المكرر □ إذا رأيت نفس بنية الكود في أكثر من مكان، يمكنك التأكد من أن برنامجك سيكون أفضل إذا وجدت طريقة لتوحيدها □ أبسط مشكلة تكرار الكود هي عندما يكون لديك نفس التعبير في طريقتين من نفس الفئة

□ قم بإجراء طريقة الاستخراج واستدعاء الكود من كلا الأماكن

□ مشكلة تكرار شائعة أخرى هي وجود نفس التعبير في فئتين فرعيتين شقيقتين □ تنفيذ طريقة السحب لأعلى

Refactoring

طريقة الاستخراج

تُستخدم طريقة الاستخراج عند وجود جزء من الكود يُمكن تجميعه. يُستخرج هذا الكود ويُحوّل إلى طريقة يُوضّح اسمها غرضها بوضوح.

Extract Method

```
void printBanner()  
{  
    System.out.println("اسم:" + _name);  
  
    System.out.println("المبلغ" + "المبلغ");  
}
```

Extract Method

إيصال طباعة باطل (مزدوج أ)

```
System.out.println // طباعة التفاصيل  
"الاسم:" System.out.println "+ _n);  
System.out.println "المبلغ"  
+ a); }
```

Extract Method

```
void printReceipt (double a)
{ printBanner(); printDetails(amount); }
```

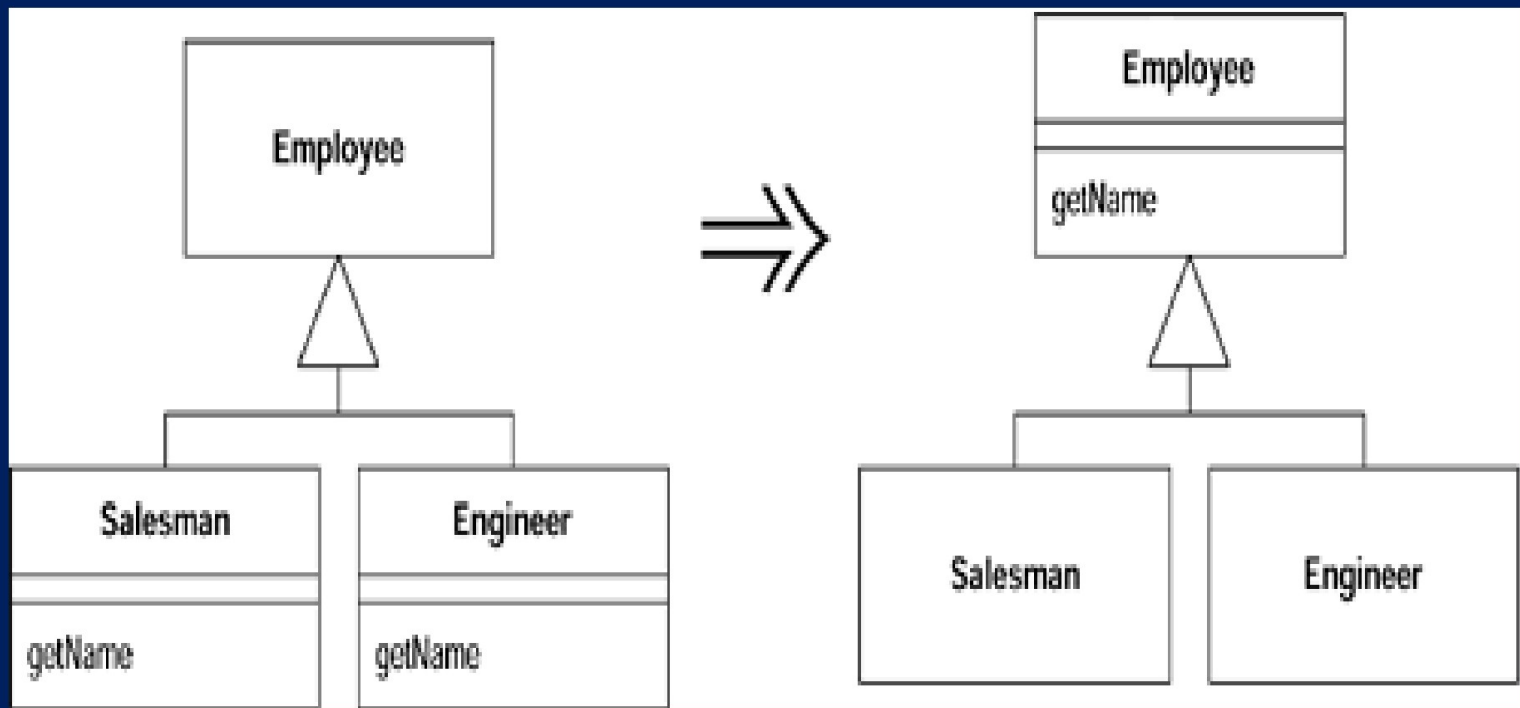
```
void printDetails
{ printBanner();
  printDetails(a); }
```

```
System.out.println ("الاسم:");
System.out.println (_name + "المبلغ" + "المبلغ");
```

Refactoring

طريقة السحب لأعلى □ إذا وجدت طرقًا في فئات فرعية مختلفة لها نتائج متطابقة، فيمكنك استخدام طريقة السحب لأعلى لنقلها إلى الفئة العليا. □ سيؤدي التخلص من هذا السلوك المكرر إلى تسهيل صيانة الكود وفهمه

Pull Up Method



Pull Up Method

```
1: public abstract class Vehicle
2: {
3:     // other methods
4: }
5:
6: public class Car : Vehicle
7: {
8:     public void Turn(Direction direction)
9:     {
10:         // code here
11:     }
12: }
13:
14: public class Motorcycle : Vehicle
15: {
16: }
17:
18: public enum Direction
19: {
20:     Left,
21:     Right
22: }
```

Pull Up Method

```
1: public abstract class Vehicle
2: {
3:     public void Turn(Direction direction)
4:     {
5:         // code here
6:     }
7: }
8:
9: public class Car : Vehicle
10: {
11: }
12:
13: public class Motorcycle : Vehicle
14: {
15: }
16:
17: public enum Direction
18: {
19:     Left,
20:     Right
21: }
```


Code Smells

-2- الطريقة الطويلة □ كلما كانت العملية أطول، كان من الصعب فهمها

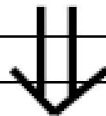
□ في أغلب الأحيان كل ما عليك فعله لتقصير طريقة ما هو استخراج الطريقة.

□ كقاعدة عامة "أكثر من شاشة واحدة أمر غير مقبول"

طويل"

Extract Method

```
void printFormatted(string text) {  
    System.out.println("Copyright (c) 1006, Siemens AG");  
    System.out.println("Author: Michael Stal");  
    printRest(text);  
}
```



```
void printFormatted(string text) {  
    printHeader();  
    printRest(text);  
}  
  
printHeader() {  
    System.out.println("Copyright (c) 1006, Siemens AG");  
    System.out.println("Author: Michael Stal");  
}
```

Code Smells

- **فئة كبيرة** □ عندما تحاول فئة ما تنفيذ مهام كثيرة، غالبًا ما يظهر ذلك على شكل عدد كبير جدًا من متغيرات المثل. عندما تحتوي الفئة على عدد كبير جدًا من متغيرات المثل، فإن تكرار الكود أمرٌ وارد.

□ الفئة التي تحتوي على قدر كبير من الكود هي أيضًا أرض خصبة للتكرار □ في كلتا الحالتين،

استخرج الفئة واستخرج

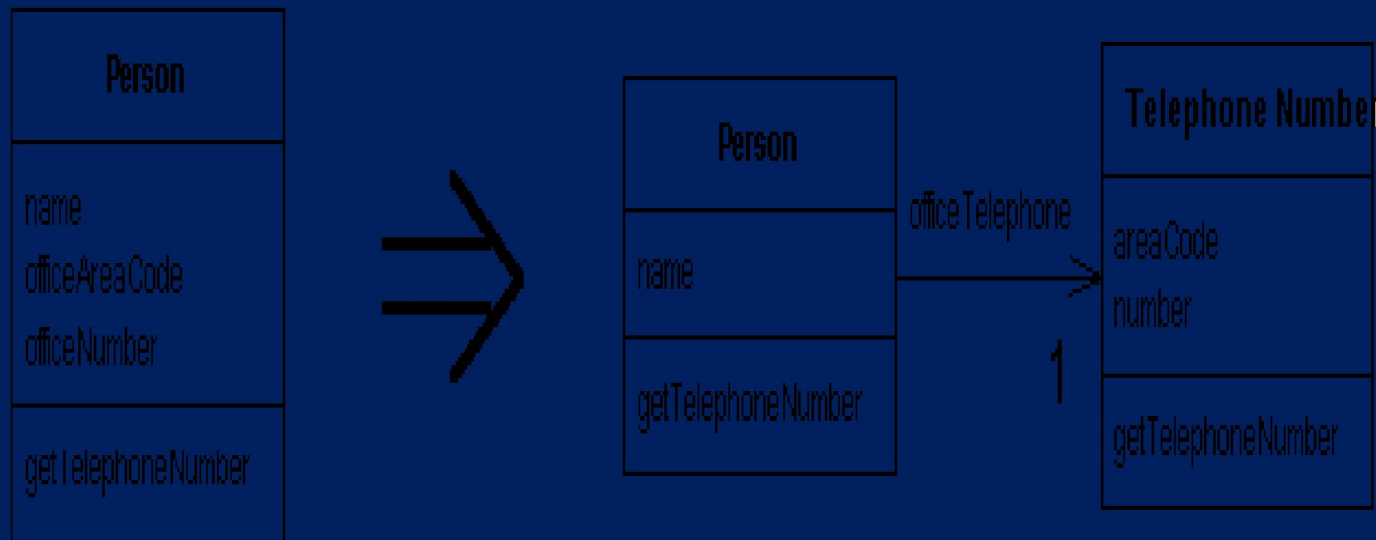
سوف تعمل الفئة الفرعية

Refactoring

استخراج الفئة

إذا وجدت أن لديك فئة واحدة تُنفَّذ عملاً ينبغي أن يُنفَّذه اثنان، فاستخدم "استخراج الفئة". ستُنشئ فئة جديدة وتنقل الدوال والسمات ذات الصلة من الفئة القديمة إلى الفئة الجديدة .

Extract Class

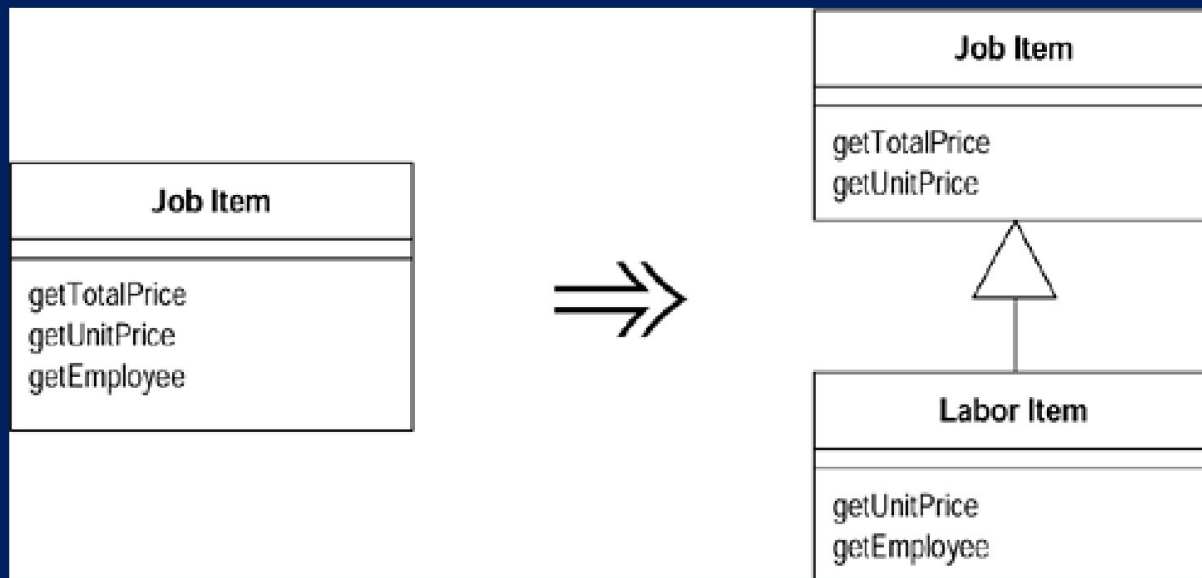


Refactoring

استخراج الفئة الفرعية

- يتم استخدامه عندما يكون للفئة بعض السلوكيات المستخدمة بعض الحالات وليس في حالات أخرى
- إنشاء فئة فرعية ترث الوظيفة المشتركة

Extract Subclass



Code Smells

-4 قائمة الملاحظات الطويلة □ من الصعب فهم قوائم الملاحظات الطويلة، لأنها غير متسقة ويصعب استخدامها لأنك تغيرها باستمرار عندما تحتاج إلى المزيد من البيانات □ استخدم استبدال المعلمة بالطريقة عندما يمكنك الحصول على البيانات في معلمة واحدة عن طريق تقديم طلب لكائن تعرفه بالفعل

Refactoring

استبدال المعلمة بالطريقة

□ يستدعي الكائن طريقة، ثم يمرر النتيجة كمعلمة للطريقة. □ يمكن للمستقبل أيضاً استدعاء هذه الطريقة.

Replace Parameter with Explicit Methods

```
void setValue(int value) {  
    if (name.equals("height")) {  
        _height = value; return; }  
}
```

{ إذا (الاسم. يساوي ("العرض")) { _العرض = القيمة؛

يعود؛
}

Replace Parameter with Explicit Methods

```
void setHeight(int arg) {  
    _الارتفاع = وسيطة؛
```

```
void setWidth (int arg) {  
    _العرض = وسيطة؛  
}
```

Code Smells

-5مميزة الحسد □ الرائحة الكلاسيكية هي طريقة تبدو أكثر اهتمامًا بفئة أخرى غير تلك التي تنتمي إليها

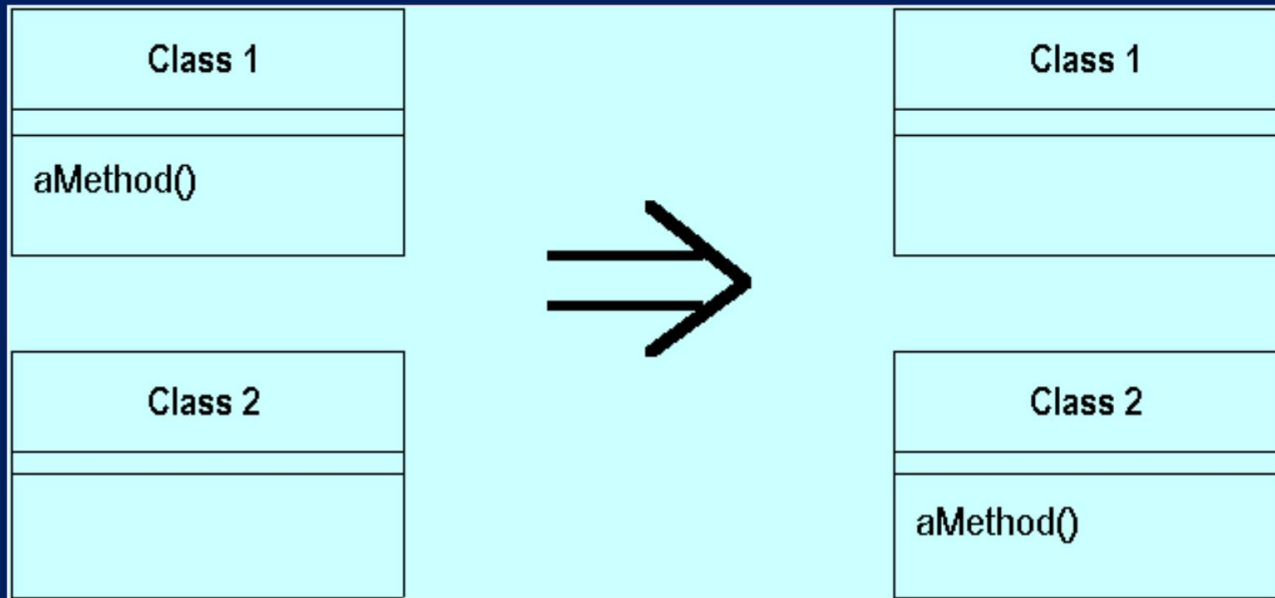
على سبيل المثال، قد تنتمي الطرق التي تستخدم فئة أخرى بشكل مكثف إلى فئة أخرى. فكّر في نقل هذه الطريقة إلى الفئة التي تُفضّلها بشدة.

□ من الواضح أن الطريقة تريد أن تكون في مكان آخر، لذا استخدم طريقة النقل للوصول إليها هناك

Refactoring

طريقة النقل

إنشاء طريقة جديدة بجسم مشابه في الفصل الذي تستخدمه أكثر



Move Method

مشروع الصف {
الشخص [] المشاركين;
}

الفئة (for(int i=0; i<p.participants.length; i++)) {
Person { int id; boolean participation(Project

} إذا {
== id) return(true); } return(false); } }
(x.participate(p)) ... إذا (p.participants[i].id

...

Move Method

مشروع الصف {
 الشخص [] المشاركين؛ المشاركة المنطقية (الشخص
 x)

{ ل i = 0؛ int i > طول المشاركين؛ } i++
 إذا (المشاركون di.[i] == x.id) ارجع (صحيح)؛ { ارجع (خطأ)؛ }

الفئة { int id; } Person

إذا ... (p.participate(x)) ...

Code Smells

6-عبارات التبديل

في أغلب الأحيان، عند رؤية عبارة تبديل، يجب عليك مراعاة تعدد الأشكال . غالبًا ما تُفَعَّل عبارة التبديل رمز النوع. أنت بحاجة إلى الطريقة أو الفئة التي تستضيف قيمة رمز النوع . استخدم طريقة الاستخراج لاستخراج عبارة التبديل، ثم طريقة النقل لإدخالها إلى الفئة التي تحتاج إلى تعدد الأشكال . استخدم استبدال الشرط بتعدد الأشكال.

Refactroing

□ استبدال الشرطية بالتعدد الأشكال

نقل كل ساق من الشرط إلى طريقة بديلة في فئة فرعية.

Replace conditional with polymorphism

الحصول على السرعة المزدوجة () {} التبديل

```
(_type) {
```

الحالة الأوروبية:

getBaseSpeed(); العودة

حالة افريقية:

إرجاع - getLoadFactor() * _numCoconuts:

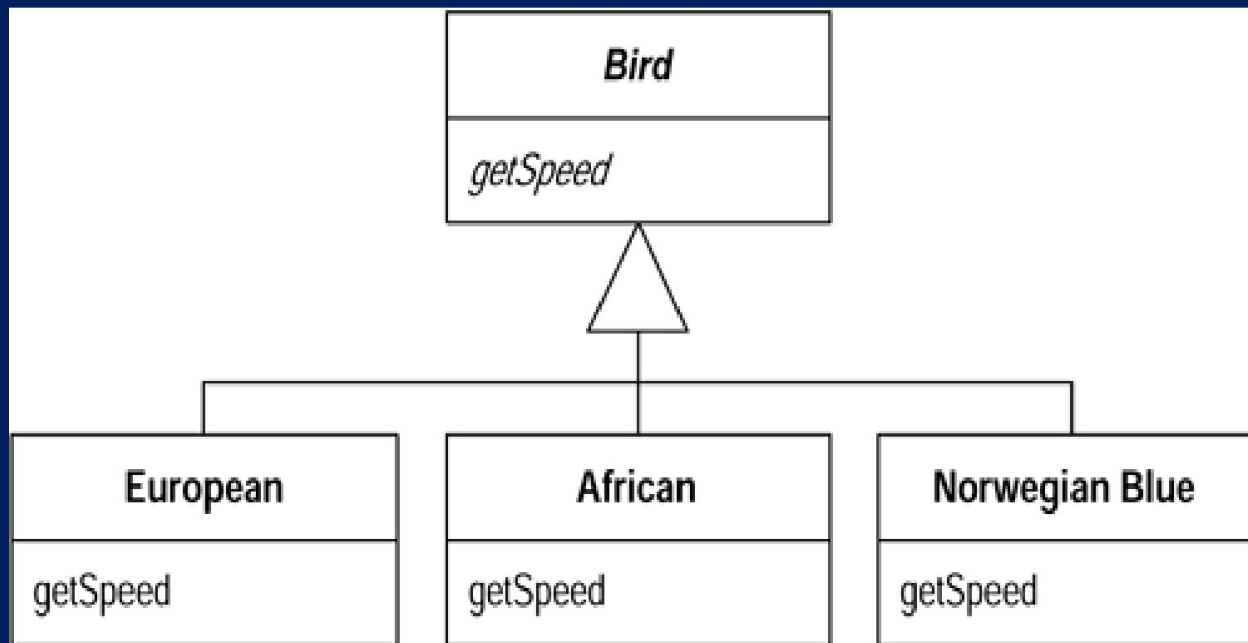
إرجاع 0 NORWEGIAN_BLUE: حالة getBaseSpeed()

```
: getBaseSpeed(_voltage);
```

```
}
```

```
}
```

Replace conditional with polymorphism



Replace conditional with polymorphism

```

EmployeeType فئة
(getTypeCode()) التوظيف tnuomAyap... int
emp) { switch
{

```

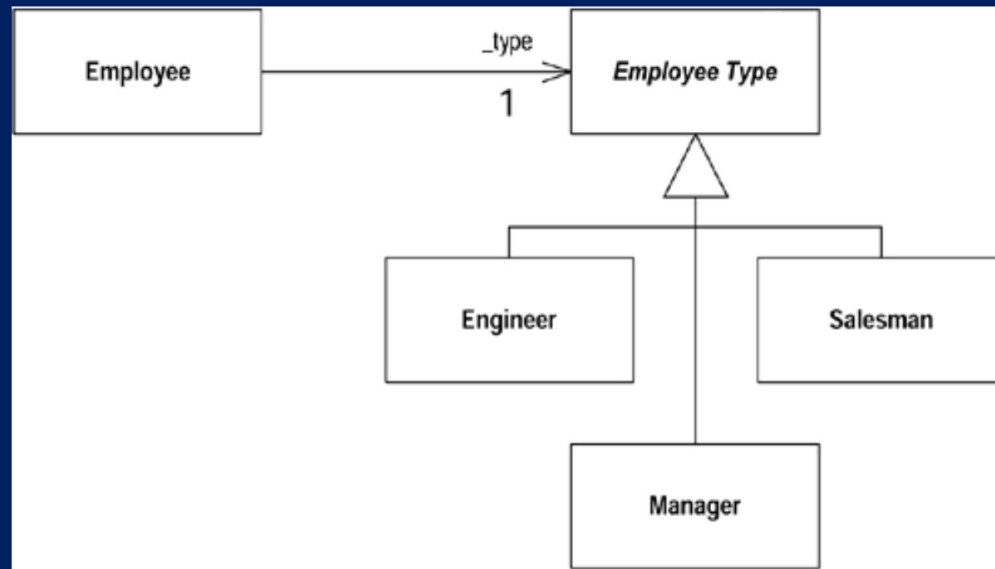
مهندس الحالة:

إرجاع emp.getMonthlySalary(); حالة البائع:

emp.getMonthlySalary() + emp.getCommission(); إرجاع
مدير الحالة:

emp.getMonthlySalary() + emp.getBonus(); إرجاع
افتراضيًا: رمي RuntimeException جديد ("موظف غير صحيح"); { {

Replace conditional with polymorphism



Replace conditional with polymorphism

فئة مهندس... tnuomAyapint (الموظف) { return emp.getMonthlySalary(); } emp (فئة بائع...)

مبلغ الدفع (الموظف)

```
{ return emp.getMonthlySalary() + emp.getCommission(); }
```

مدير الفئة...

مبلغ الدفع (الموظف)

```
{ return emp.getMonthlySalary() + emp.getBonus(); }
```

إعلان طريقة الفئة العليا المجردة:

EmployeeType... الفئة

ملخص tnuomAyapint (الموظف): emp)

Bad Smells

-7 الفئة الكسولة □ كل فئة تقوم بإنشائها تكلف المال والوقت

الحفاظ عليها وفهمها

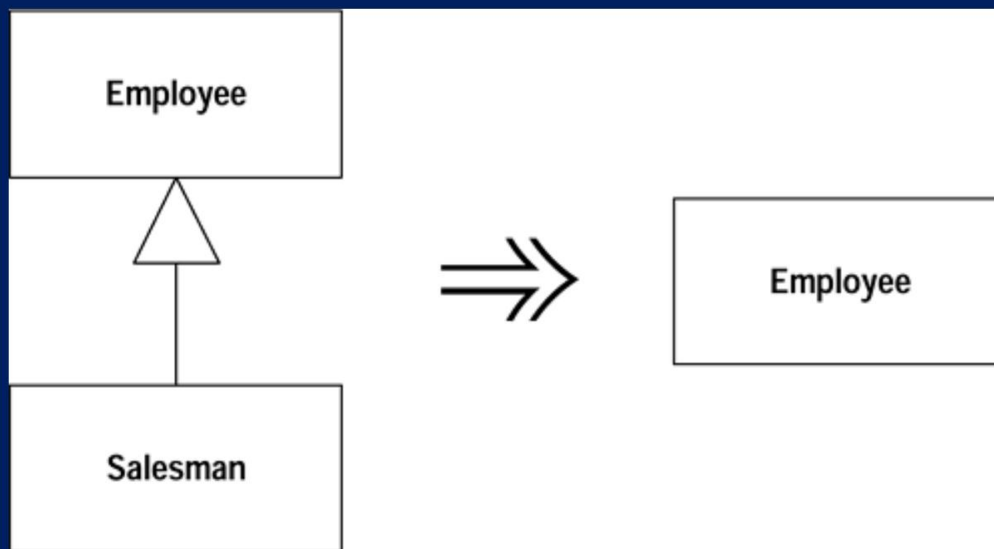
□ يجب التخلص من الفئة التي لا تحمل وزنها

□ إذا كان لديك فئات فرعية لا تقوم بما يكفي، فحاول استخدام **Hierarchy** ، **Collapse** ويجب أن تخضع المكونات عديمة الفائدة تقريبًا لـ

فئة مضمنة

Refactoring

انهيار التسلسل الهرميا يوجد اختلاف كبير بين الطبقة العليا والطبقة الفرعية .

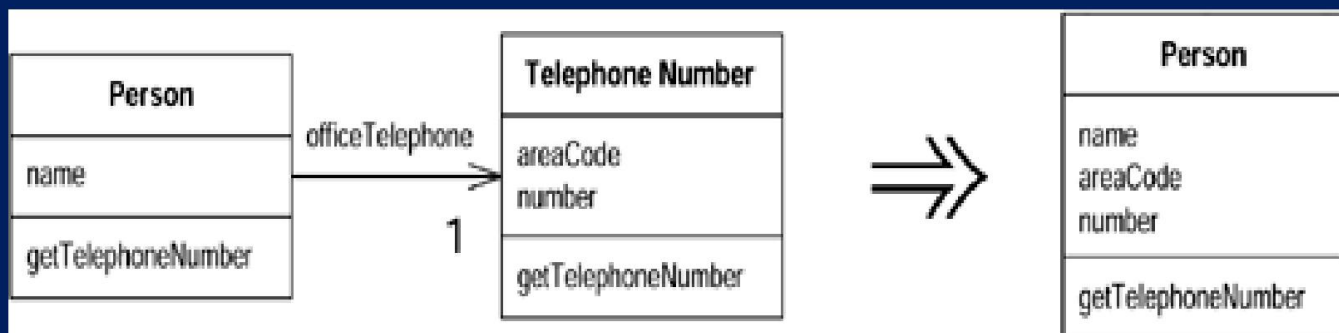


Refactoring

الفئة المضمنة

الفئة المضمنة هي عكس فئة الاستخراج. استخدم الفئة المضمنة إذا لم تعد الفئة تؤدي وظيفتها ولا ينبغي أن تكون موجودة بعد الآن.

غالبًا ما تكون هذه هي النتيجة لإعادة الهيكلة التي تنقل مسؤوليات أخرى خارج الفصل بحيث لا يتبقى سوى القليل



Bad Smells

-8 الكود الميت

هل لديك كتل من الأكواد البرمجية غير المستخدمة أو المُعلّق عليها في مشروعك؟ لا تتركها هناك تحسبًا لحاجتك إليها، بل اختبرها للتأكد من عدم حاجتك إليها الآن وتخلص منها. إن لم تفعل، فستجعل مطورًا آخر يقضي وقتًا طويلًا في قراءتها وفهم فائدتها. احذف الأكواد البرمجية غير المستخدمة بلا رحمة.

Bad Smells

-الرقم السحري

استبدال الرقم السحري بثابت رمزي □ قم بإنشاء ثابت، ثم قم بتسميته وفقاً للمعنى، ثم استبدل الرقم به

Refectory

ثابت مزدوج تحويل الجنيهات إلى كيلو (جنيه مزدوج) { إرجاع
جنيه / } ؛ 2.2

نهائي ثابت مزدوج

معامل تحويل الكيلو ؛ 2.2 =

ثابت مزدوج تحويل الجنيهات إلى كيلو (جنيه مزدوج) { إرجاع
جنيهات / } ؛ KILO_CONVERSION_FACTOR